

Running Spark on YARN

Support for running on [YARN \(Hadoop NextGen\)](#) was added to Spark in version 0.6.0, and improved in subsequent releases.

Launching Spark on YARN

Ensure that `HADOOP_CONF_DIR` or `YARN_CONF_DIR` points to the directory which contains the (client side) configuration files for the Hadoop cluster. These configs are used to write to HDFS and connect to the YARN ResourceManager. The configuration contained in this directory will be distributed to the YARN cluster so that all containers used by the application use the same configuration. If the configuration references Java system properties or environment variables not managed by YARN, they should also be set in the Spark application's configuration (driver, executors, and the AM when running in client mode).

There are two deploy modes that can be used to launch Spark applications on YARN. In `yarn-cluster` mode, the Spark driver runs inside an application master process which is managed by YARN on the cluster, and the client can go away after initiating the application. In `yarn-client` mode, the driver runs in the client process, and the application master is only used for requesting resources from YARN.

Unlike in Spark standalone and Mesos mode, in which the master's address is specified in the `--master` parameter, in YARN mode the ResourceManager's address is picked up from the Hadoop configuration. Thus, the `--master` parameter is `yarn-client` or `yarn-cluster`. To launch a Spark application in `yarn-cluster` mode:

```
$ ./bin/spark-submit --class path.to.your.Class --master yarn-cluster [options] <app jar> [app options]
```

For example:

```
$ ./bin/spark-submit --class org.apache.spark.examples.SparkPi \  
  --master yarn-cluster \  
  --num-executors 3 \  
  --driver-memory 4g \  
  --executor-memory 2g \  
  --executor-cores 1 \  
  --queue thequeue \  
  lib/spark-examples*.jar \  
  10
```

The above starts a YARN client program which starts the default Application Master. Then SparkPi will be run as a child thread of Application Master. The client will periodically poll the Application Master for status updates and display them in the console. The client will exit once your application has finished running. Refer to the “Debugging your Application” section below for how to see driver and executor logs.

To launch a Spark application in `yarn-client` mode, do the same, but replace `yarn-cluster` with `yarn-client`. To run `spark-shell`:

```
$ ./bin/spark-shell --master yarn-client
```

Adding Other JARs

In `yarn-cluster` mode, the driver runs on a different machine than the client, so `sparkContext.addJar` won't work out of the box with files that are local to the client. To make files on the client available to `sparkContext.addJar`, include them with the `--jars` option in the launch command.

```
$ ./bin/spark-submit --class my.main.Class \  
  --master yarn-cluster \  
  --jars my-other-jar.jar,my-other-other-jar.jar  
  my-main-jar.jar  
  app_arg1 app_arg2
```

Preparations

Running Spark-on-YARN requires a binary distribution of Spark which is built with YARN support. Binary distributions can be downloaded from the Spark project website. To build Spark yourself, refer to [Building Spark](#).

Configuration

Most of the configs are the same for Spark on YARN as for other deployment modes. See the [configuration page](#) for more information on those. These are configs that are specific to Spark on YARN.

Debugging your Application

In YARN terminology, executors and application masters run inside “containers”. YARN has two modes for handling container logs after an application has completed. If log aggregation is turned on (with the `yarn.log-aggregation-enable` config), container logs are copied to HDFS

and deleted on the local machine. These logs can be viewed from anywhere on the cluster with the “yarn logs” command.

```
yarn logs -applicationId <app ID>
```

will print out the contents of all log files from all containers from the given application. You can also view the container log files directly in HDFS using the HDFS shell or API. The directory where they are located can be found by looking at your YARN configs (`yarn.nodemanager.remote-app-log-dir` and `yarn.nodemanager.remote-app-log-dir-suffix`). The logs are also available on the Spark Web UI under the Executors Tab. You need to have both the Spark history server and the MapReduce history server running and configure `yarn.log.server.url` in `yarn-site.xml` properly. The log URL on the Spark history server UI will redirect you to the MapReduce history server to show the aggregated logs.

When log aggregation isn't turned on, logs are retained locally on each machine under `YARN_APP_LOGS_DIR`, which is usually configured to `/tmp/logs` or `$HADOOP_HOME/logs/userlogs` depending on the Hadoop version and installation. Viewing logs for a container requires going to the host that contains them and looking in this directory. Subdirectories organize log files by application ID and container ID. The logs are also available on the Spark Web UI under the Executors Tab and doesn't require running the MapReduce history server.

To review per-container launch environment, increase `yarn.nodemanager.delete.debug-delay-sec` to a large value (e.g. 36000), and then access the application cache through `yarn.nodemanager.local-dirs` on the nodes on which containers are launched. This directory contains the launch script, JARs, and all environment variables used for launching each container. This process is useful for debugging classpath problems in particular. (Note that enabling this requires admin privileges on cluster settings and a restart of all node managers. Thus, this is not applicable to hosted clusters).

To use a custom log4j configuration for the application master or executors, there are two options:

- upload a custom `log4j.properties` using `spark-submit`, by adding it to the `--files` list of files to be uploaded with the application.
- add `-Dlog4j.configuration=<location of configuration file>` to `spark.driver.extraJavaOptions` (for the driver) or `spark.executor.extraJavaOptions` (for executors). Note that if using a file, the `file:` protocol should be explicitly provided, and the file needs to exist locally on all the nodes.

Note that for the first option, both executors and the application master will share the same log4j configuration, which may cause issues when they run on the same node (e.g. trying to write to the same log file).

If you need a reference to the proper location to put log files in the YARN so that YARN can properly display and aggregate them, use `spark.yarn.app.container.log.dir` in your `log4j.properties`. For example, `log4j.appender.file_appender.File=${spark.yarn.app.container.log.dir}/spark.log`. For streaming application, configuring `RollingFileAppender` and setting file location to YARN's log directory will avoid disk overflow caused by large log file, and logs can be accessed using YARN's log utility.

Spark Properties

Property Name	Default	Meaning
<code>spark.yarn.am.memory</code>	512m	Amount of memory to use for the YARN Application Master in client mode. In the same format as JVM memory strings (e.g. 512m, 2g). In cluster mode, use <code>spark.driver.memory</code> instead.
<code>spark.driver.cores</code>	1	Number of cores used by the driver in YARN cluster mode. Since the driver is run in the same JVM as the YARN Application Master in cluster mode, this also controls the cores used by the YARN AM. In client mode, use <code>spark.yarn.am.cores</code> to control the number of cores used by the YARN AM instead.
<code>spark.yarn.am.cores</code>	1	Number of cores to use for the YARN Application Master in client mode. In cluster mode, use <code>spark.driver.cores</code> instead.
<code>spark.yarn.am.waitTime</code>	100s	In 'yarn-cluster' mode, time for the application master to wait for the SparkContext to be initialized. In 'yarn-client' mode, time for the application master to wait for the driver to connect to it.
<code>spark.yarn.submit.file.replication</code>	The default HDFS replication (usually 3)	HDFS replication level for the files uploaded into HDFS for the application. These include things like the Spark jar, the app jar, and any distributed cache files/archives.
<code>spark.yarn.preserve.staging.files</code>	false	Set to true to preserve the staged files (Spark jar, app jar, distributed cache files) at the end of the job rather than delete them.
<code>spark.yarn.scheduler.heartbeat.interval-ms</code>	3000	The interval in ms in which the Spark application master heartbeats into the YARN ResourceManager. The value is capped at half the value of YARN configuration for the expiry interval (<code>yarn.am.liveness-monitor.expiry-interval-ms</code>).
<code>spark.yarn.scheduler.initial-allocation.interval</code>	200ms	The initial interval in which the Spark application master eagerly heartbeats to the YARN ResourceManager when there are pending container allocation requests. It should be no larger than <code>spark.yarn.scheduler.heartbeat.interval-ms</code> . The allocation interval is doubled on successive eager heartbeats if pending containers still exist until <code>spark.yarn.scheduler.heartbeat.interval-ms</code> is reached.

<code>spark.yarn.max.executor.failures</code>	numExecutors * 2, with minimum of 3	The maximum number of executor failures before failing the application
<code>spark.yarn.historyServer.address</code>	(none)	The address of the Spark history server (i.e. host.com:18080). The address should not contain a scheme (http://). Defaults to not being set since the history server is an optional service. This address is given to the YARN ResourceManager when the Spark application finishes to link the application from the ResourceManager UI to the Spark history server UI. For this property, YARN properties can be used as variables, and these are substituted by Spark at runtime. For eg, if the Spark history server runs on the same node as the YARN ResourceManager, it can be set to <code>`\${hadoopconf-yarn.resourcemanager.hostname}:18080`</code> .
<code>spark.yarn.dist.archives</code>	(none)	Comma separated list of archives to be extracted into the working directory of each executor.
<code>spark.yarn.dist.files</code>	(none)	Comma-separated list of files to be placed in the working directory of each executor.
<code>spark.executor.instances</code>	2	The number of executors. Note that this property is incompatible with <code>spark.dynamicAllocation.enabled</code> . If both <code>spark.dynamicAllocation.enabled</code> and <code>spark.executor.instances</code> are specified, dynamic allocation is turned off and the specified number of <code>spark.executor.instances</code> is used.
<code>spark.yarn.executor.memoryOverhead</code>	executorMemory * 0.10, with minimum of 384	The amount of off heap memory (in megabytes) to be allocated per executor. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the executor size (typically 6-10%).
<code>spark.yarn.driver.memoryOverhead</code>	driverMemory * 0.10, with minimum of 384	The amount of off heap memory (in megabytes) to be allocated per driver in cluster mode. This is memory that accounts for things like VM overheads, interned strings, other native overheads, etc. This tends to grow with the container size (typically 6-10%).
<code>spark.yarn.am.memoryOverhead</code>	AM memory * 0.10, with minimum of 384	Same as <code>spark.yarn.driver.memoryOverhead</code> , but for the Application Master in client mode.
<code>spark.yarn.am.port</code>	(random)	Port for the YARN Application Master to listen on. In YARN client mode, it is used to communicate between the Spark driver running on a gateway and the Application Master running on YARN. In YARN cluster mode, it is used for the dynamic executor feature, where it handles the kill from the scheduler backend.
<code>spark.yarn.queue</code>	default	The name of the YARN queue to which the application is submitted.
<code>spark.yarn.jar</code>	(none)	The location of the Spark jar file, in case overriding the default location is desired. By default, Spark on YARN will use a Spark jar installed locally, but the Spark jar can also be in a world-readable location on HDFS. This allows YARN to cache it on nodes so that it doesn't need to be distributed each time an application runs. To point to a jar on HDFS, for example, set this configuration to <code>"hdfs://some/path"</code> .
<code>spark.yarn.access.namenodes</code>	(none)	A list of secure HDFS namenodes your Spark application is going to access. For example, <code>`spark.yarn.access.namenodes=hdfs://nn1.com:8032,hdfs://nn2.com:8032`</code> . The Spark application must have access to the namenodes listed and Kerberos must be properly configured to be able to access them (either in the same realm or in a trusted realm). Spark acquires security tokens from each of the namenodes so that the Spark application can access those remote HDFS clusters.
<code>spark.yarn.appMasterEnv. [EnvironmentVariableName]</code>	(none)	Add the environment variable specified by <code>EnvironmentVariableName</code> to the Application Master process launched on YARN. The user can specify multiple of these and to set multiple environment variables. In <code>`yarn-client`</code> mode this controls the environment of the SPARK driver and in <code>`yarn-cluster`</code> mode it only controls the environment of the executor launcher.
<code>spark.yarn.containerLauncherMaxThreads</code>	25	The maximum number of threads to use in the application master for launching executor containers.
<code>spark.yarn.am.extraJavaOptions</code>	(none)	A string of extra JVM options to pass to the YARN Application Master in client mode. In cluster mode, use <code>`spark.driver.extraJavaOptions`</code> instead.
<code>spark.yarn.am.extraLibraryPath</code>	(none)	Set a special library path to use when launching the application master

client mode.

<code>spark.yarn.maxAppAttempts</code>	<code>yarn.resourcemanager.am.max-attempts</code> in YARN	The maximum number of attempts that will be made to submit the application. It should be no larger than the global number of max attempt in the YARN configuration.
<code>spark.yarn.submit.waitAppCompletion</code>	<code>true</code>	In YARN cluster mode, controls whether the client waits to exit until the application completes. If set to true, the client process will stay alive reporting the application's status. Otherwise, the client process will exit submission.
<code>spark.yarn.executor.nodeLabelExpression</code>	<code>(none)</code>	A YARN node label expression that restricts the set of nodes executors be scheduled on. Only versions of YARN greater than or equal to 2.6 support node label expressions, so when running against earlier version this property will be ignored.
<code>spark.yarn.keytab</code>	<code>(none)</code>	The full path to the file that contains the keytab for the principal specified above. This keytab will be copied to the node running the Application Master via the Secure Distributed Cache, for renewing the login tickets the delegation tokens periodically.
<code>spark.yarn.principal</code>	<code>(none)</code>	Principal to be used to login to KDC, while running on secure HDFS.
<code>spark.yarn.config.gatewayPath</code>	<code>(none)</code>	A path that is valid on the gateway host (the host where a Spark application is started) but may differ for paths for the same resource in other nodes in the cluster. Coupled with <code>spark.yarn.config.replacementPath</code>, this is used to support clusters with heterogeneous configurations, so that Spark can correctly launch remote processes. The replacement path normally will contain a reference to some environment variable exported by YARN (and, thus, visible to Spark containers). For example, if the gateway node has Hadoop libraries installed on <code>/disk1/hadoop</code>, and the location of the Hadoop install is exported by YARN as the <code>HADOOP_HOME</code> environment variable, setting this value to <code>/disk1/hadoop</code> and the replacement path to <code>\$HADOOP_HOME</code> will make sure that paths used to launch remote processes properly reference the local YARN configuration.
<code>spark.yarn.config.replacementPath</code>	<code>(none)</code>	See <code>spark.yarn.config.gatewayPath</code> .

Important notes

- Whether core requests are honored in scheduling decisions depends on which scheduler is in use and how it is configured.
- In `yarn-cluster` mode, the local directories used by the Spark executors and the Spark driver will be the local directories configured for YARN (Hadoop YARN config `yarn.nodemanager.local-dirs`). If the user specifies `spark.local.dir`, it will be ignored. In `yarn-client` mode, the Spark executors will use the local directories configured for YARN while the Spark driver will use those defined in `spark.local.dir`. This is because the Spark driver does not run on the YARN cluster in `yarn-client` mode, only the Spark executors do.
- The `--files` and `--archives` options support specifying file names with the `#` similar to Hadoop. For example you can specify: `--files localtest.txt#appSees.txt` and this will upload the file you have locally named `localtest.txt` into HDFS but this will be linked to by the name `appSees.txt`, and your application should use the name as `appSees.txt` to reference it when running on YARN.
- The `--jars` option allows the `sparkContext.addJar` function to work if you are using it with local files and running in `yarn-cluster` mode. It does not need to be used if you are using it with HDFS, HTTP, HTTPS, or FTP files.